



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: June 12, 2026

Contents

Objetivos de Aprendizaje	3
Marco Teórico	3
Topología de prueba	3
Desarrollo	4
Generación y Comparación de Hashes	4
Hashing en Python	6
Vulnerabilidad y Protección con Hash	6
Investigación	7
Conclusiones	7
Referencias Bibliográficas	7

List of Figures

1	Guardado de Mensaje “Seguridad en redes 2025” en Documento	4
2	Ejecución de Algoritmo de Hash “md5sum”	4
3	Ejecución de Algoritmo de Hash “sha1sum”	5
4	Ejecución de Algoritmo de Hash “sha256sum”	5
5	Envío de Documento desde Kali a Ubuntu	5
6	Encriptado de Documento	6
7	Ejecución de Script Python de Ejercicio 2	6
8	Ejecución de Script Python de Ejercicio 3	7

List of Tables

Objetivos de Aprendizaje

Al finalizar esta práctica, usted será capaz de:

1. Comprender el funcionamiento de las funciones hash criptográficas.
2. Utilizar comandos y herramientas en Linux para calcular hashes.
3. Verificar la integridad de archivos mediante valores hash.
4. Aplicar el hashing en scripts de Python para mejorar la seguridad del código.
5. Identificar prácticas inseguras que pueden evitarse usando funciones hash.
6. Investigar y aplicar conceptos de autenticidad, integridad y verificación mediante hashes.

Marco Teórico

Las funciones hash criptográficas son funciones matemáticas que transforman datos de tamaño arbitrario en una cadena de longitud fija. Estas funciones tienen las siguientes propiedades:

- Unidireccionalidad: no se puede obtener el dato original a partir del hash.
- Determinismo: la misma entrada siempre produce la misma salida.
- Resistencia a colisiones: es computacionalmente difícil encontrar dos entradas distintas con el mismo hash.
- Avalanche: un pequeño cambio en la entrada produce un hash totalmente distinto.

En la práctica, los algoritmos hash son ampliamente usados para:

- Verificar la integridad de archivos.
- Guardar contraseñas de forma segura.
- Firmas digitales y certificados.
- Evitar la ejecución de código no autorizado.

Algunos algoritmos comunes incluyen MD5, SHA-1 y SHA-256, entre otros. Aunque algunos (como MD5 o SHA-1) ya no se consideran seguros para aplicaciones críticas, siguen siendo útiles para fines educativos o de verificación básica.

Topología de prueba

Se utilizarán las siguientes máquinas virtuales:

- Ubuntu Desktop: estación de trabajo del estudiante.
- Ubuntu Server: servidor de archivos.
- Kali Linux: entorno para validación de vulnerabilidades y pruebas de seguridad.

Desarrollo

Generación y Comparación de Hashes

En Ubuntu Desktop:

1. Crear un archivo de texto:

```
echo "Seguridad en redes 2025" > documento.txt
```

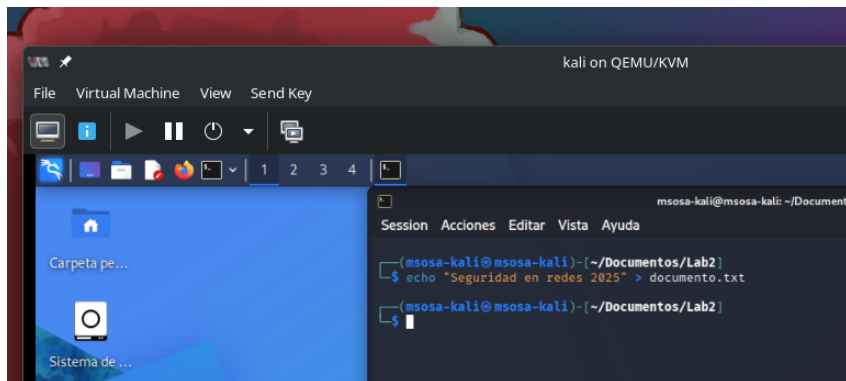


Figure 1: Guardado de Mensaje “Seguridad en redes 2025” en Documento

2. Generar el hash del archivo con diferentes algoritmos:

```
md5sum documento.txt
```

```
sha1sum documento.txt
```

```
sha256sum documento.txt
```

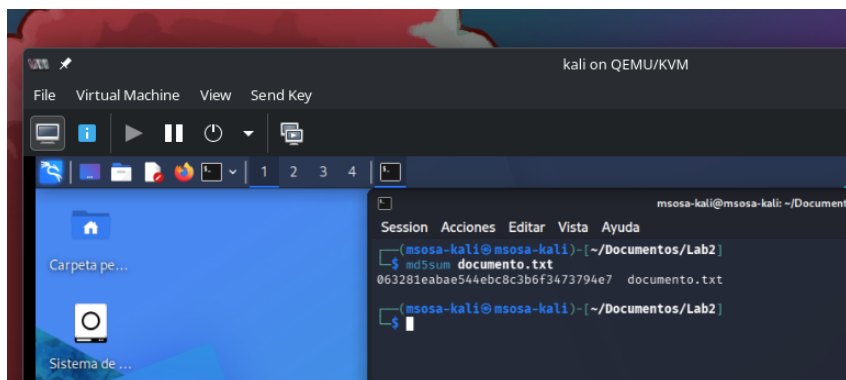


Figure 2: Ejecución de Algoritmo de Hash “md5sum”

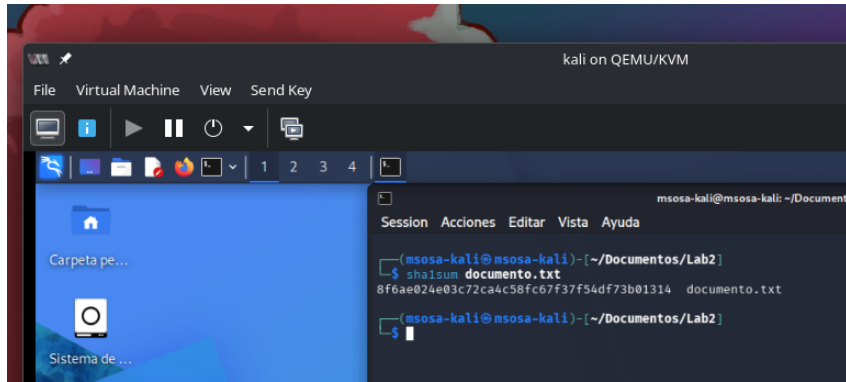


Figure 3: Ejecución de Algoritmo de Hash “sha1sum”

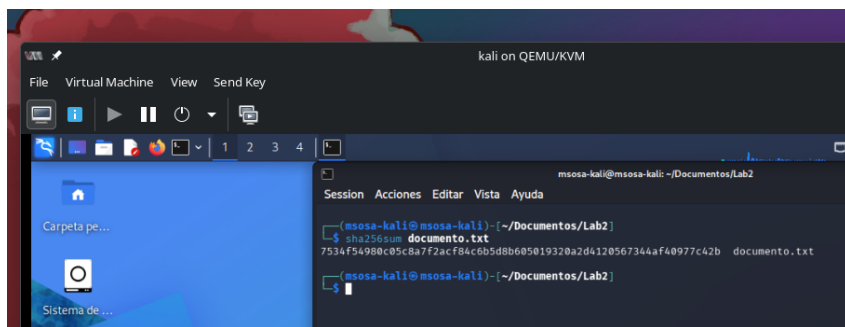


Figure 4: Ejecución de Algoritmo de Hash “sha256sum”

3. Modificar el contenido del archivo (por ejemplo, cambiar una letra), volver a generar los hashes y observar cómo cambian.
4. Transferir el archivo al servidor Ubuntu Server y verificar que no fue alterado:

```
scp documento.txt usuario@IP_SERVER:/home/usuario/ssh usuario@IP_SERVER
```

```
sha256sum documento.txt
```

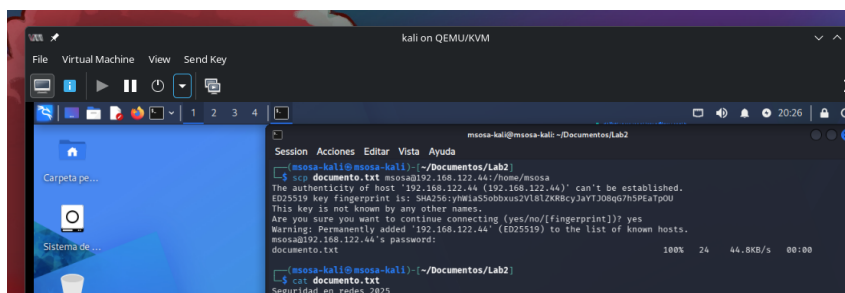


Figure 5: Envío de Documento desde Kali a Ubuntu

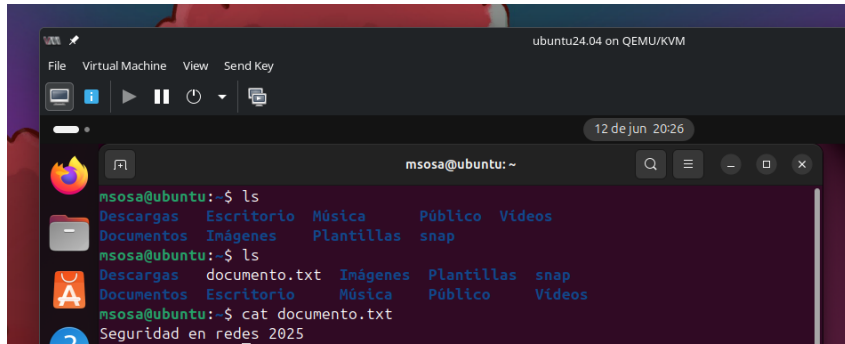


Figure 6: Encriptado de Documento

Hashing en Python

1. Crear un script en Ubuntu Desktop llamado hash_script.py:

```
import hashlib

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

passwd = input("Introduce una contraseña: ")
print("Hash SHA-256:", hash_password(passwd))
```

2. Ejecutar el script y probar con diferentes contraseñas para ver cómo cambia el hash.

```
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej2.py
Introduce una contraseña: Mateo
Hash SHA-256: 8272dce0d7192d4599a7ee498066f8e7c36dcffe1a7c70f15854fd228fdfa25
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej2.py
Introduce una contraseña: Hola
Hash SHA-256: e633f4fc79badea1dc5db970cf397c8248bac47cc3acf9915ba60b5d76b0e88f
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej2.py
Introduce una contraseña: Test
Hash SHA-256: 532eaabd9574880dbf76b9b8cc00832c20a6ec113d682299550d7a6e0f345e25
```

Figure 7: Ejecución de Script Python de Ejercicio 2

Vulnerabilidad y Protección con Hash

Código vulnerable en Python que compara contraseñas en texto plano:

```
passwords = ["admin123", "qwerty"]
user_input = input("Contraseña: ")
if user_input in passwords:
    print("Acceso concedido")
else:
    print("Acceso denegado")
```

```
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej3.py
Contraseña: Hola
Acceso denegado
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$ python ej3.py
Contraseña: admin123
Acceso concedido
msosa@fedora: ~/Documents/ESPE/seguridad-30808/P2/Laboratorios/2$
```

Figure 8: Ejecución de Script Python de Ejercicio 3

Problema: el almacenamiento y la comparación de contraseñas en texto plano representa una grave vulnerabilidad de seguridad.

Investigación

Los estudiantes deberán investigar y aplicar una solución para almacenar y comparar contraseñas de forma segura usando funciones hash. Deberán responder:

1. ¿Qué riesgos presenta almacenar contraseñas en texto plano?

Almacenar contraseñas en texto plano permite que cualquier persona con acceso a la base de datos pueda verlas y utilizarlas directamente.

2. ¿Qué ventajas ofrece el uso de hashing?

El hashing protege las contraseñas al almacenarlas como valores irreversibles, evitando que se conozca su contenido original.

3. ¿Qué algoritmos de hash son recomendables para el almacenamiento de contraseñas?

Los algoritmos más recomendados son bcrypt, Argon2 y PBKDF2, ya que están diseñados específicamente para proteger contraseñas.

4. ¿Cómo implementar una comparación segura de contraseñas en Python?

La comparación segura se implementa almacenando el hash de la contraseña y verificando la contraseña ingresada contra dicho hash mediante bibliotecas como `bcrypt` o `argon2`.

Conclusiones

1. Las funciones hash permiten asegurar la integridad y autenticidad de datos en entornos digitales.
2. Son herramientas esenciales en la seguridad informática, tanto en redes como en desarrollo de software.
3. El uso de hashes puede prevenir vulnerabilidades comunes como el almacenamiento inseguro de contraseñas.
4. Entornos como Linux y lenguajes como Python ofrecen múltiples utilidades para aplicar criptografía hash de forma eficiente y didáctica.

Referencias Bibliográficas

1. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*. Pearson.

2. Paar, C., & Pelzl, J. (2010). *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer.
3. The Open Web Application Security Project (OWASP). (2024). *Password Storage Cheat Sheet*. <https://cheatsheetseries.owasp.org>
4. Python Software Foundation. (2024). *Python 3 Standard Library: hashlib*. <https://docs.python.org/3/library/hashlib.html>
5. Linux Man Pages. md5sum, sha1sum, sha256sum